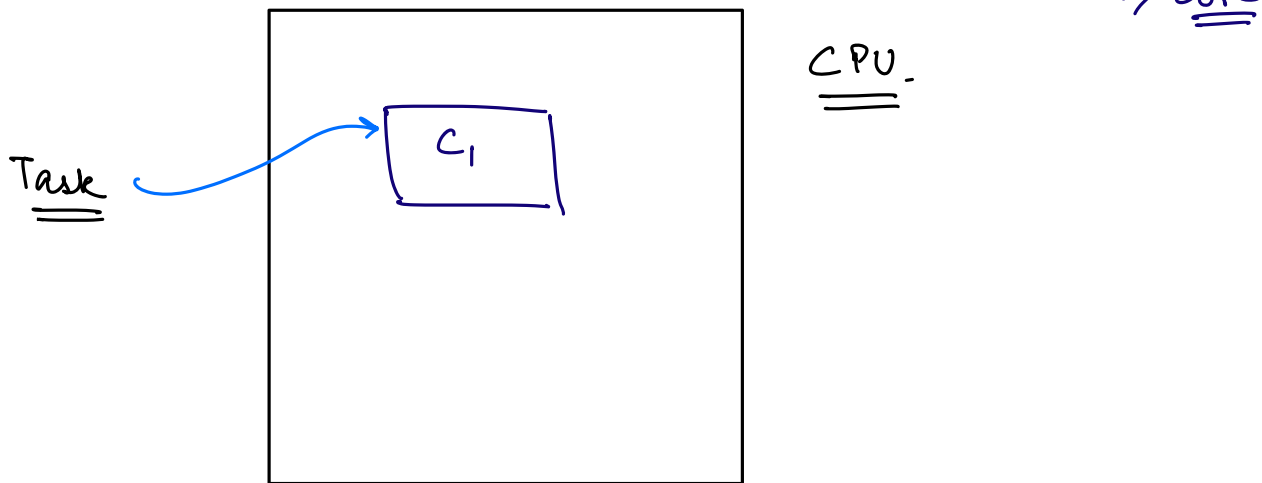


Agenda.

- Intro to Multithreading
- Hello World from new thread.
- Print no's from 1 to 100
- Executor framework
- Thread Pool

⇒



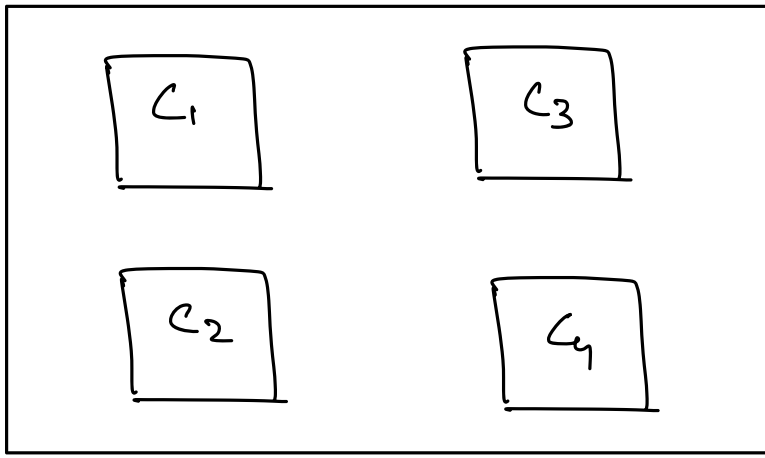
⇒ CPU executes the task with the help of Cores

CPU

└→ Quad Core $\Rightarrow$ 4

└→ Octa Core $\Rightarrow$ 8

—
—



$\Rightarrow$ Single Core CPU.

$\hookrightarrow$ One task at a time.

1 Core $\Rightarrow$ 1 GHz

$\Rightarrow$ 10^9 operations/sec.

$T_1 \Rightarrow 10^6$ operations.

$10^9 \Rightarrow$ 1 sec

$\frac{1}{10^9} \Rightarrow \frac{1}{10^9}$ sec

$10^6 \Rightarrow \frac{10^6}{10^9}$ sec

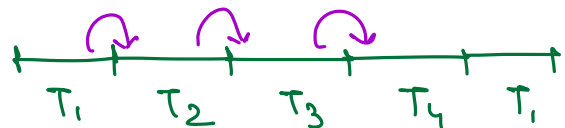
$\Rightarrow 10^{-3}$ sec

$\Rightarrow$ 1 ms.

T_2

T_3

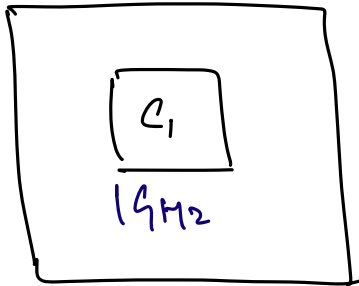
T_4



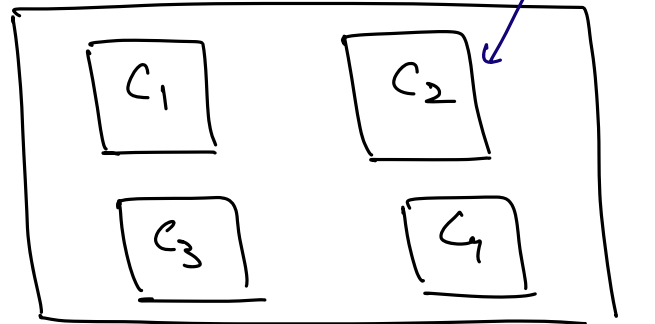
$\Rightarrow$ Context Switching.

CPU

Single Core



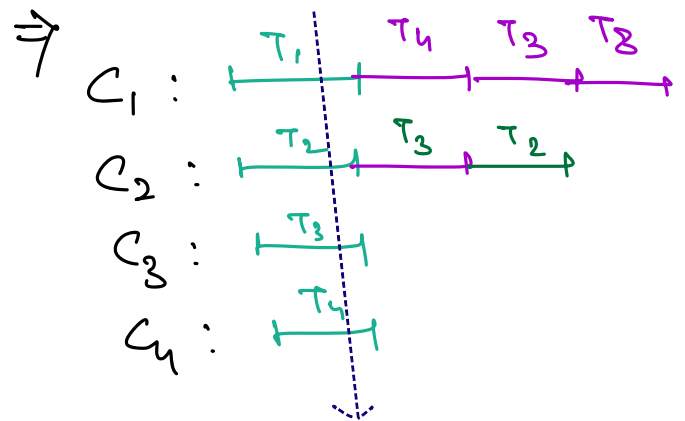
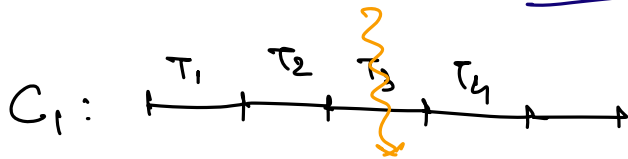
Multi Core



⇒ Only one task a time

⇒ Context switches ↑

⇒ Concurrent System.



⇒ Context switches ↓

⇒ Parallelism.

Quad Core $\begin{cases} 4 \Rightarrow 1\text{GHz} \\ 4 \Rightarrow \underline{\underline{4\text{GHz}}}. \end{cases}$

GPU.

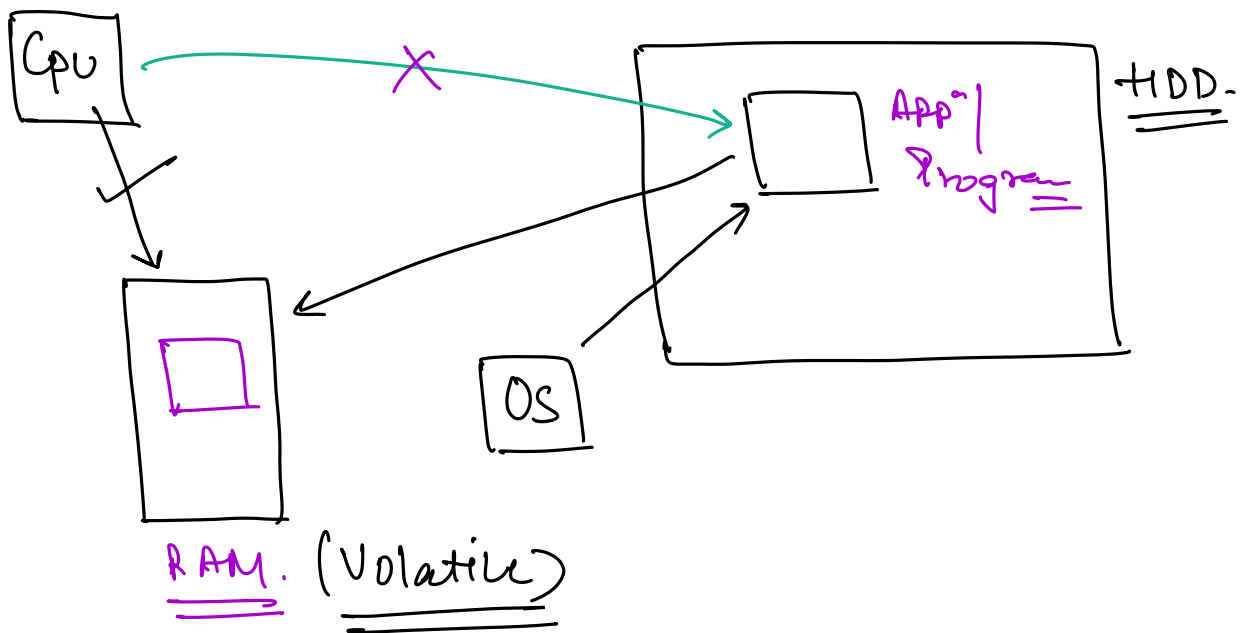
↳ 1000's of Core.

2.5 MHz

Download & Install an Appⁿ.

↓
Executable file.

- .exe
- .apk
- .dmg



CPU can't directly talk to HDD, because of the speed difference.

MS Word | Document Processing App

- Auto save
- Spell Checker
- Grammar Check
- Auto Suggestion

I ma teaching
multithreading
in JAVA.

⇒ Parallelly.

```
int a = 10;  
int b = 20  
int c = a + b;  
Print(c)
```



Program (HDD)



OS



RAM

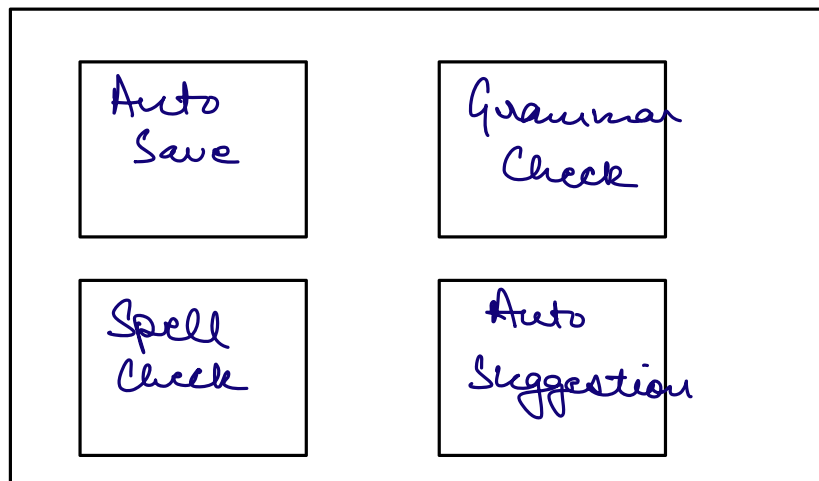


CPU

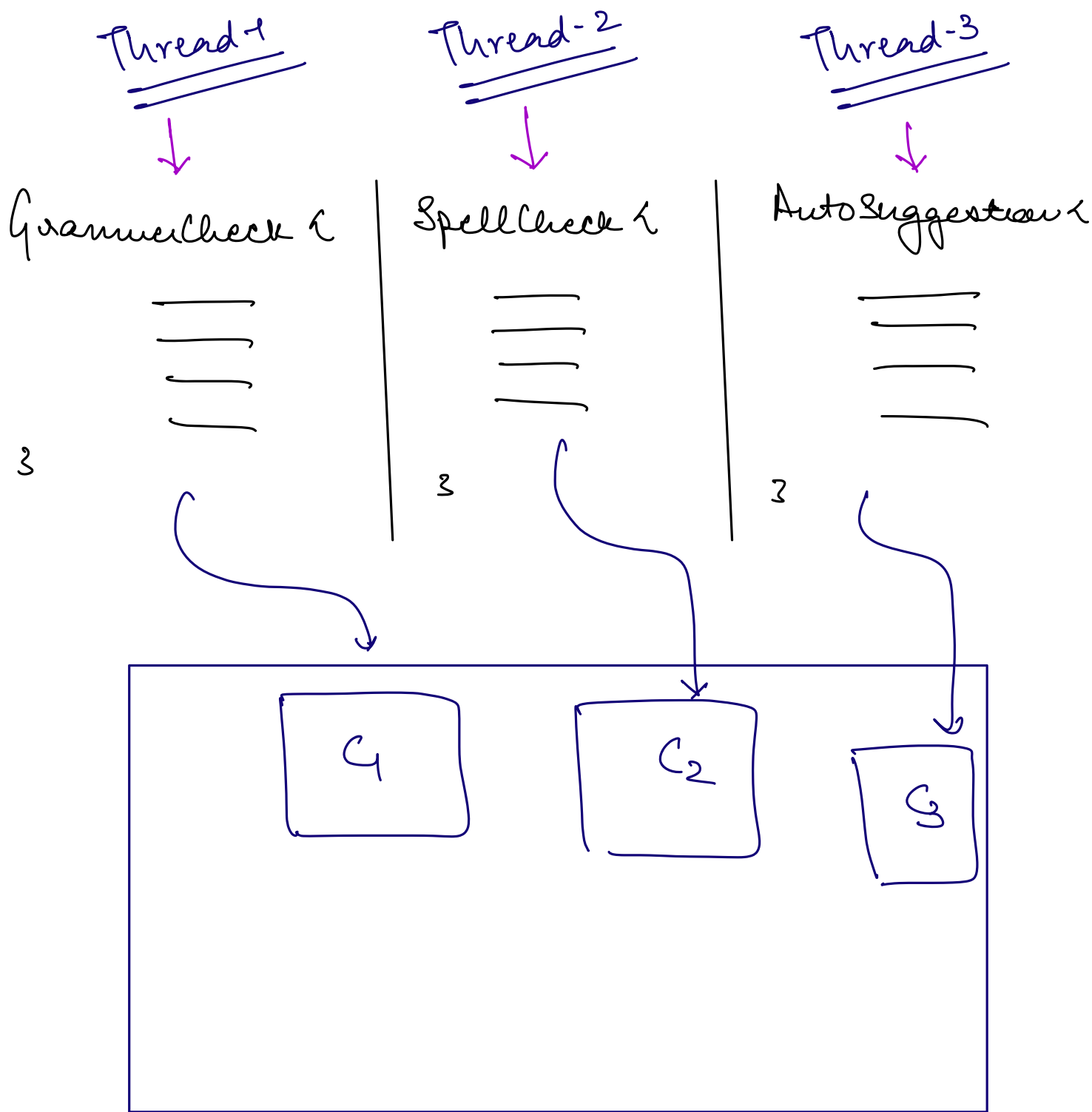


Process.

→ Program in execution.



Quad
Core



⇒ JAVA Program.

↳ print (Hello world)
↳ Main thread
↳ default thread.

Print Hello World from a new thread.

Steps.

I) Create a task class that we want to execute in a separate thread.

Class HelloWorldPrinter {

}

II) Make the task class implement Runnable interface.

Class HelloWorldPrinter implements Runnable {

}

Functional
Interface.

III) Override the run() method

Task
↓

Class HelloWorldPrinter implements Runnable {

public void run() {

System.out.println("Hello world");

}

3

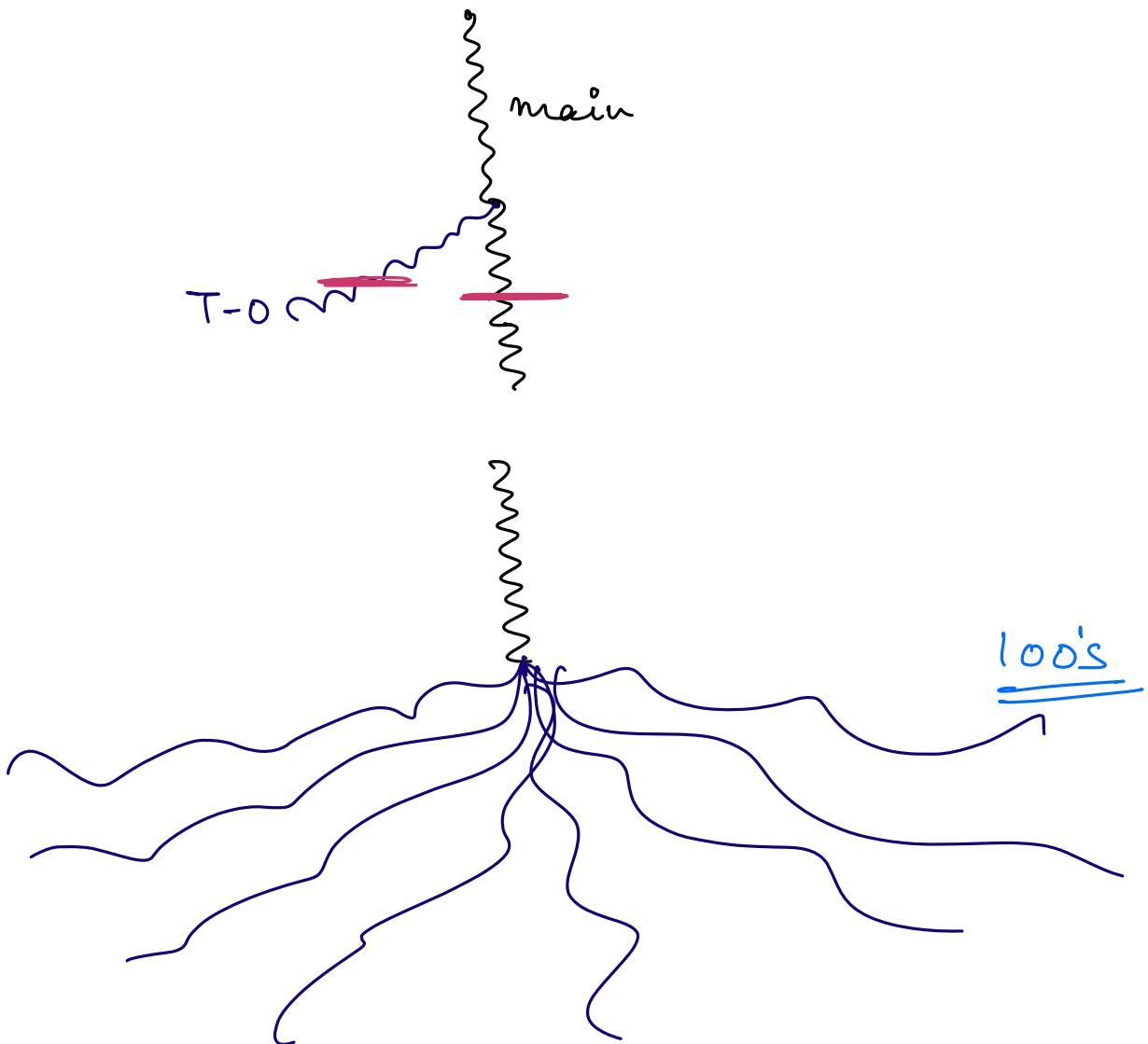
IV) Create a thread object assign the task to that.

```
HelloWorldPrinter task = new HelloWorldPrinter();
```

```
Thread (t) = new Thread(task);
```

V) Start the thread.

```
t.start();
```



Print numbers from 1 to 100, each from a separate thread.

1 → T-0
2 → T-1
3 → T-2
4
⋮
⋮
100 → T-99

Class NumberPrinter implements Runnable {
int number;

NumberPrinter(int number) {
this.number = number;

3

public void run() {
 sout(number, "...");

}

3

main() {

for (i = 1; i ≤ 100; i++) {

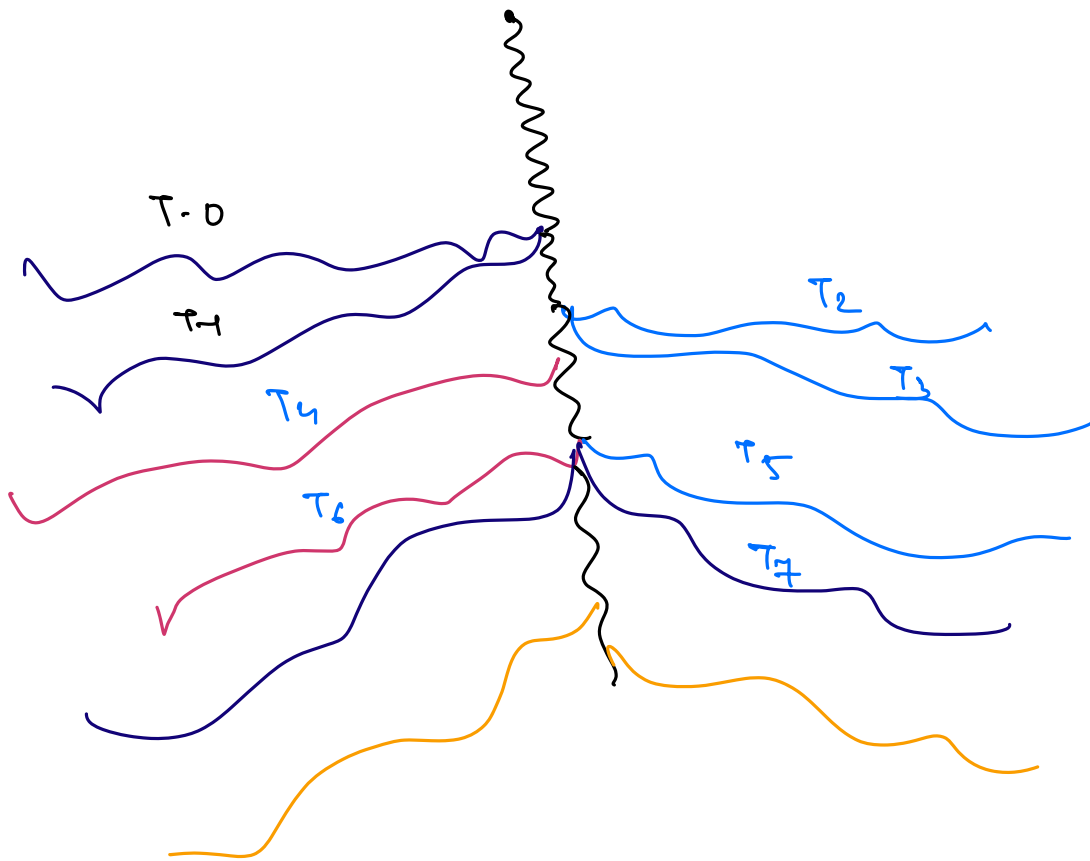
NumberPrinter task = new NumberPrinter(i);

Thread t = new Thread(task)

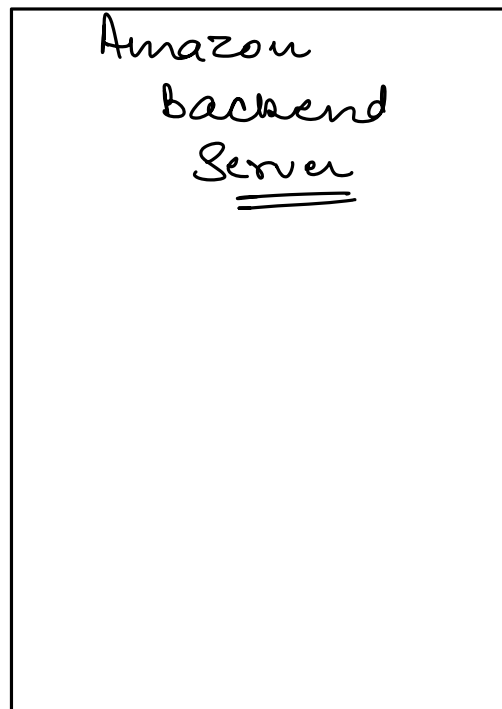
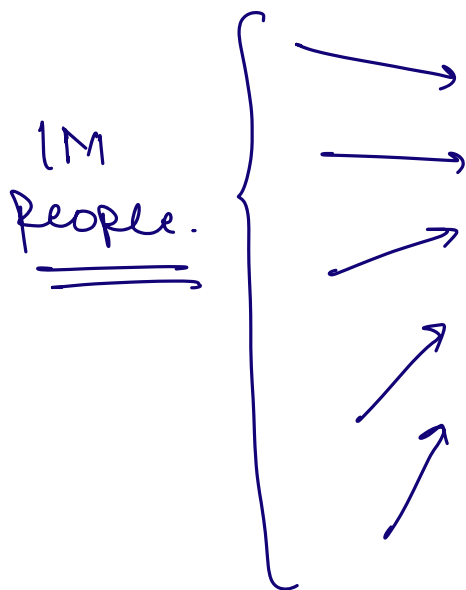
t.start();

}

10



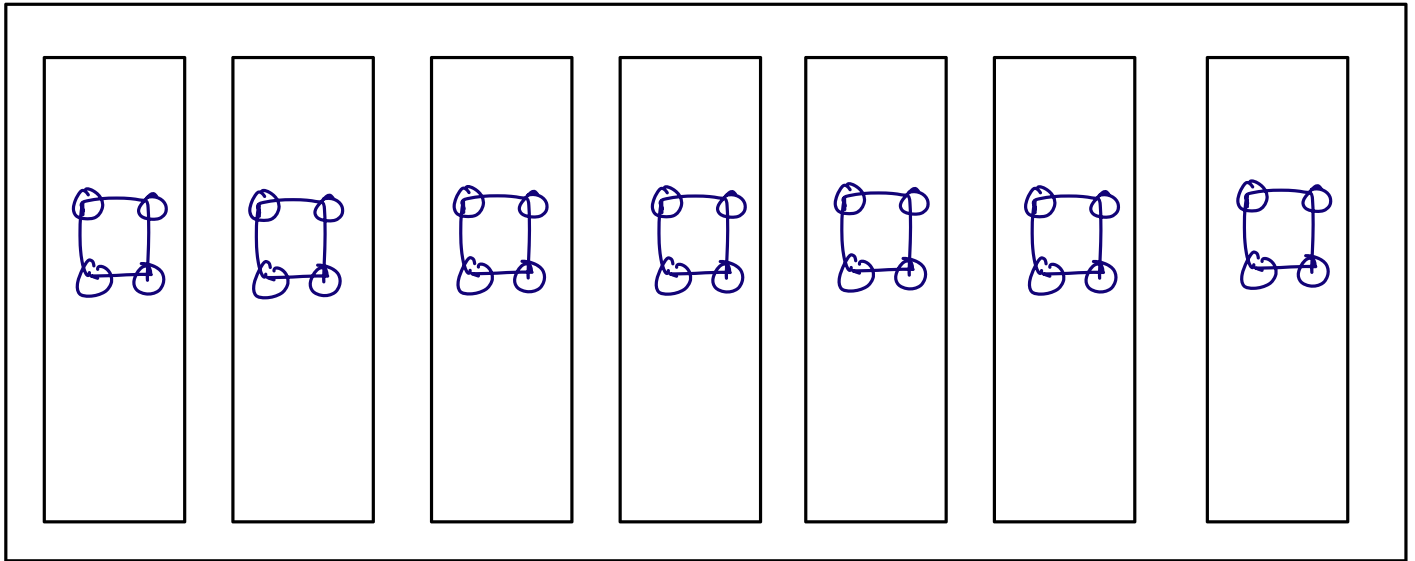
$\Rightarrow$ 100 threads.



Test.

↳ Multiple Production Lines.

One car can be produced
at a time.

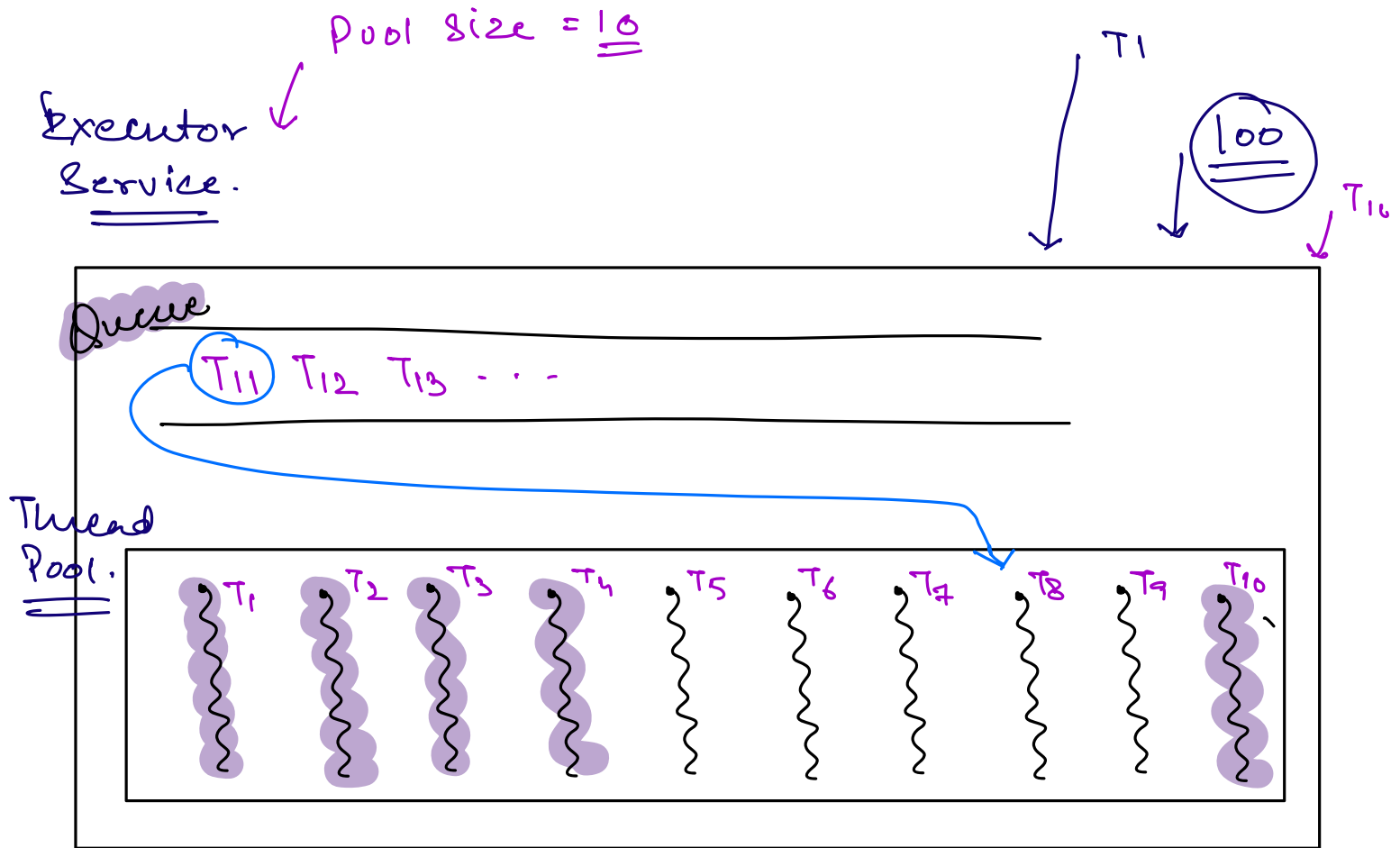


⇒ 1000 Cars.

⇒ 1000 Production Lines.

⇒ Optimise the utilization of existing production lines.

⇒ Executor Framework & Thread Pool.



⇒ Efficient management & utilization of threads.